

Pre-PGI: What We Found in Your SAP System

What happens between delivery and goods issue, which parts of it can actually be called from outside SAP, and why we recommend one interface rather than three.

SYSTEMS EXAMINED	DATE	CHANGES MADE	BASIS
DS4 / 200 QS4 / 700	18 Aug 2026	None	Live system reads only

RECOMMENDATION

The pre-PGI span — shipment, shipment cost, cost release — should be **one API call**, not three. Not for elegance: because **four of the six steps have no callable SAP interface at all**, so custom development is required either way.

Every figure and document number in this document is a live read from your system. Nothing is estimated.

Contents

1 **How this was established**

Access, evidence standard, tools

2 **What SAP actually does before PGI**

Six steps, a real chain, what your status fields mean

3 **What is callable, and what is not**

Services, BAPIs, internal functions, the verdict

4 **What we found that is not currently known**

10,496 stranded documents · cost type · the PGI question

5 **Comparison with the v1.8 specification**

Corrections needed and the gap we can now close

6 **Our recommendation**

One API — and the conditions we state with it

7 **What remains open, and what we need**

Questions, the test-data blocker, the decision we need

8 **Summary**

Seven findings and their consequences

1 How this was established

1.1 Systems, access and what was touched

System	Client	Access	Changes made
DS4 — development	200	Write-authorised	None
QS4 — quality	700	Read-only	None

No document was created, changed, posted or reversed in either system. Our findings come from reading your service definitions, your function-module interfaces, your repository index and your existing documents.

1.2 What we treated as evidence

We separated what is *proven* from what is *assumed*, because that difference decides how much weight this can carry.

We treat as proven	We do not
A live response from your system, with its status recorded	A service appearing in a catalogue
An interface read directly from the system	A sample payload generated from that interface
A document chain traced field by field	Two timestamps close together, assumed to be one transaction
A true row count	A partial sample treated as the whole population

TWO DISTINCTIONS WE APPLY THROUGHOUT

Callable is not the same as supported. A function can be technically callable while SAP makes no commitment to keep its interface stable across upgrades. Three of the interfaces here are in that category.

A successful technical response is not business authorisation. It proves the service is reachable — nothing about whether the business operation will succeed.

1.3 The tools used

Tool	What it told us
Gateway Client	Which OData services are live, and their exact contracts
Service Builder (SEGW)	The design-time catalogue your specification was built from
Function Builder (SE37)	Real interfaces, support status and remote-call flags for every BAPI in scope
Where-Used analysis	The decisive tool — it found what actually performs each step, and who calls it
Data browser and dictionary	What your data actually looks like, and what your status fields really mean

2 What SAP actually does before PGI

2.1 The six steps

The portal shows one action. SAP performs **six independently committed steps**. Each is a separate database commit — once one completes, it cannot be undone by a rollback.

#	Step	Stored in	Proof it completed
1	Shipment created	VTTK / VTTP	Shipment number
2	Cost document created	VFKK / VFKP	Cost document number
3	Calculated	VFKP	Calculation status complete
4	Account assigned	VFKP	Account assignment complete
5	Settled / transferred	VFKP + Materials Management	Freight PO and service entry created
6	Goods issue (PGI)	MKPF / LIKP	Material document, movement 601

2.2 A real completed chain from your system

Delivery 9004953084 , 17 August 2026:

- 10:26:23** ● **Delivery 9004953084 created**
Route E36881
- 11:11:28** ● **Shipment 1100605471 created**
Carrier 0013000730 · 484 KM · vehicle HR55AZ3800
- 11:11:28** ● **Cost document 1100608871 created and calculated**
7,460.00 INR
- 11:11:30** ● **Cost settled into Materials Management**
Freight PO 6000105831 · service entry 1003382363
- 11:11:31** ● **Goods issue — material document 4918168265**
- 11:11:32** ● **Billing 1108024764**

TWO SECONDS

Shipment created to cost settled took **11:11:28 to 11:11:30**. This is a fast sequential chain, not a long-running process that needs external checkpoints between steps.

2.3 What your status fields actually mean

A CORRECTION THAT MATTERS

The field `VFKK-STFRE` has been understood as the “cost release” status. Read from the SAP dictionary, it is the **account assignment** status. And **blank** means “not relevant”, not “not done” — so a blank field never indicated a missed step.

Field	What it really means
<code>STBER</code>	Calculation status
<code>STFRE</code>	Account assignment status
<code>STABR</code>	Transfer / settlement status

There is **no release field** on the cost document header. What the business calls “cost release” is technically the **transfer of freight cost into Materials Management** — which is why completing it produces a freight purchase order and a service entry sheet.

2.4 How the documents link

CORRECTION

It has been recorded that the cost document number equals the shipment number. **It does not.**

Cost document	Actual shipment
1100608871	1100605471
2100005093	2100005099 — <i>not</i> 2100005093

The number ranges are similar enough to look interchangeable. Any reporting or reconciliation built on “cost document equals shipment” will silently produce wrong results.

3 What is callable, and what is not

3.1 The OData services

Seven services are live in your development system and all return their contract successfully.

Service	Serves
API_MATERIAL_DOCUMENT_SRV	Goods receipt
API_OUTBOUND_DELIVERY_SRV v2	Delivery create, picking, goods issue, delivery reads
API_BILLING_DOCUMENT_SRV	Billing read-back
API_PURCHASEORDER_PROCESS_SRV	Stock transport orders
API_MATERIAL_STOCK_SRV	Book stock
MMIM_MATDOC_SRV	Material document read-back
SD_CUSTOMER_INVOICES_CREATE	Billing creation

THE DELIVERY SERVICE MUST BE VERSION 2

Version 1 exists under the identical name and lacks the batch-split operations. It would activate cleanly and then fail on first real use.

BILLING CANNOT BE CREATED THROUGH THE BILLING API

All eight of its data sets are marked not-creatable. Billing creation must go through a different service. Any design assuming otherwise needs correcting.

3.2 The BAPIs

Fifteen interfaces were read from your system. All are callable remotely. **Three are not released** — SAP makes no commitment to keep their interface stable across upgrades. If any is adopted, that carries a named upgrade risk.

Interface	Released	Role
BAPI_SHIPMENT_CREATE	Yes	The one pre-PGI step with a proper standard API
BAPI_SHIPMENT_COST_ESTIMATE	No	Freight estimate — creates no document
BAPI_SHIPMENT_CHANGE	No	Not proven to perform cost release
BAPI_OUTB_DELIVERY_CHANGE	No	Delivery quantity change alternative
Goods movement, delivery create, PO create, billing	Yes	Other APIs in the portfolio

YOUR TRANSPORT FIELDS HAVE A HOME

Vehicle number, driver name and mobile, LR/GR number and date, and the three weights are customer fields on the **shipment creation** structure. The specification currently parks them as belonging to an unidentified route. They belong to the shipment creation call.

Two practical notes. `BAPI_SHIPMENT_CREATE` has no test mode — its first execution creates a real shipment. And the freight estimate is not a delivery-only call: it requires a full in-memory shipment to be constructed, not just a delivery number and a transporter.

3.3 What performs the middle steps

Steps 2 to 5 are performed by internal SAP function modules. We identified all of them by name:

Step	SAP function module
Create cost document	SD_SCDS_CREATE
Calculate	SD_SCD_ITEM_CALCULATE
Account assign	SD_SCD_ITEM_ACCT_ASSIGNMENT
Settle / release	SD_SCDS_RELEASE
Persist	SD_SCDS_SAVE

THE TERMINOLOGY, CONFIRMED BY SAP ITSELF

`SD_SCDS_RELEASE` 's own SAP description translates as “**transfer of freight cost items**” — not “release”. This is why completing it creates a purchase order and a service entry sheet in Materials Management.

SAP REFUSES TO SPLIT TWO OF THESE STEPS

`SD_SCDS_CREATE` calls `SD_SCDS_SAVE` internally. Creating and persisting the cost document is a **single SAP operation**. Exposing them as two endpoints would invent a boundary that does not exist.

3.4 The verdict

Step	Standard callable API?
1 · Shipment create	YES
— · Freight estimate	Callable, but not a released interface
2 · Cost document create	NO
3 · Calculate	NO
4 · Account assign	NO
5 · Settle / release	NO
6 · Goods issue	YES

THE CONCLUSION THAT DRIVES OUR RECOMMENDATION

Four of the six pre-PGI steps have no callable SAP interface.

Custom SAP development is therefore **mandatory**, regardless of how many endpoints we expose. This is a property of the system, not a design preference. “Use standard APIs instead” is not an available option for this span.

Method note: we state this as a finding from the searches performed — function-module name patterns and full-scope where-used analysis, each with a control to prove the search itself was working. We do not claim no such interface could exist anywhere.

4 What we found that is not currently known

4.1 10,496 shipment-cost documents are stranded

Full count across your entire cost-document table:

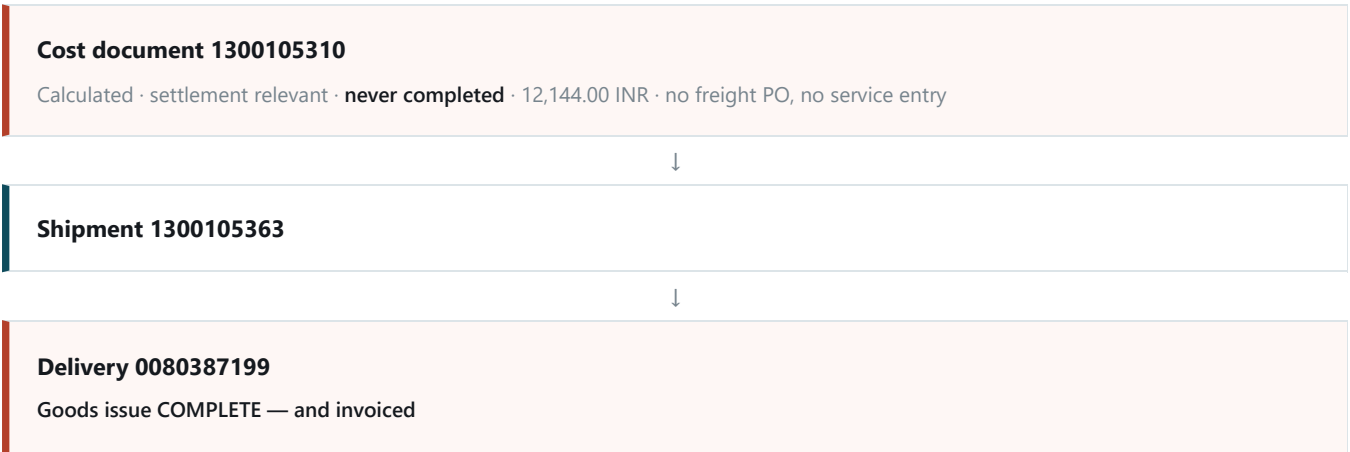
52.4% settled		47.4% not relevant	
3,021,454 fully transferred	2,737,318 not relevant	10,496 stranded (0.18%)	
Settlement status	Meaning	Count	Share
Fully transferred	Completed correctly	3,021,454	52.4%
Not relevant	Settlement never applied	2,737,318	47.4%
Relevant, never transferred	Should have completed — did not	10,496	0.18%
Total		5,769,268	

0.18% is a small proportion, and we want to be straight about that. This is not a system in crisis. But it is the number that matters, for this reason:

This 0.18% is the **best case**. It comes from code running inside SAP, in the same program context, with no network between the steps, tuned over years of production use. It still leaves 10,496 documents stranded. An integration that splits these steps across a network boundary, with an orchestrator that cannot read the cost tables, will not do better.

At roughly 4,000 logistics calls per day, 0.18% is about **7 documents a day, 2,600 a year** — each with freight cost calculated but never posted to Materials Management. No freight purchase order, no service entry sheet, no accrual cleared.

A traced example



The delivery shipped and was billed. The freight cost never posted. That is what these 10,496 documents look like.

4.2 Cost type is a required input, and it is not constant

Any call that creates a shipment cost document must supply a **cost type**. Four types behave differently in your system:

Cost type	Observed behaviour
Z001	Completes fully — freight PO and service entry created
Z003	Strands — this is the 10,496 population
Z006	Calculates; settlement not relevant
Y003	Calculated only

NEW INFORMATION

On 17 August, cost release was described as “a tick inside the transaction, not a separate BAPI”. That is accurate in the user interface. But the **callable** form requires a cost type to be chosen and supplied — and the current specification has no field for it. This needs a decision: is it derived from the shipment, or does the caller send it?

4.3 Settlement is not technically required before PGI

This was an open question. It is now answered, and the answer is not what was assumed.

PROVEN FROM YOUR DATA

SAP does not block goods issue on unsettled freight cost. The example above proves it: settlement was relevant and incomplete, and the delivery was goods-issued and invoiced anyway.

What this means. “Release before goods issue” is a **business rule, not a SAP constraint**. If you want it enforced, the interface must enforce it — SAP will not. Equally, an orchestration that waits for settlement is imposing a constraint SAP does not impose. That should be a deliberate decision rather than an inherited assumption.

4.4 This pattern is already implemented in your system

The internal function that creates cost documents is called from **eight places, all of them custom code written for this landscape** — and from no SAP standard program. One of them is a function module with this interface:

Input	
Shipment number	mandatory
Cost type	mandatory
Commit flag	optional

One shipment in, cost document posted, commit under the caller's control. **This is the shape we are proposing, and it already exists and runs here** — a useful feasibility proof.

TWO THINGS MUST BE SAID ABOUT IT

1. It belongs to a different programme — outside C&F scope — so we cite it as proof the pattern works, **not** as something CNF can simply reuse.
2. It returns **nothing**: no document number, no error. It cannot tell its caller what it created or why it failed. That is the one thing our version must do differently.

5 Comparison with the v1.8 specification

5.1 Corrections required

Current specification says	Your system says
VFKK-STFRE is the cost release status	It is account assignment; blank means “not relevant”
Cost document number equals shipment number	False — they are different numbers
Batch split call takes 4 parameters	5 — a unit parameter is missing; the call would fail
Billing can be created via the billing API	Not possible — read-only for creation
Transport fields have no identified home	They are fields on the shipment-create structure
Cost type not mentioned	It is a mandatory input and varies by scenario
Stage-B route “unidentified”	Identified by name — and none of it is callable

5.2 The gap in the specification

The API-03 request section has field-level detail for every stage **except** the shipment and cost stage, which carries only a note that the route is unidentified.

That gap can now be closed. The request contract is the shipment-create structure including your vehicle, driver, LR and weight fields, plus the delivery reference, plus a decision on cost type. We will produce this as a correction report rather than editing the approved workbook.

5.3 Status of the twelve APIs

We hold a deliberately strict standard: an operation counts as validated only when a real input was executed and the result reconciled to an SAP document. **On that standard, two operations are validated.** Most of the rest are blocked not by SAP, but by the absence of test data in the development system — see 7.2.

API	Status
API-01 · Goods receipt	Contract confirmed; blocked by missing test data
API-02 · Create delivery	Contract confirmed; blocked by missing predecessors
API-03 · Invoice chain	Pre-PGI half requires custom development — section 6
API-04 · Freight estimate	Interface not released; input larger than assumed
API-05 · Stock	Correct service, correctly read-only
API-06 / 07 · Statutory	Existing route; boundary not yet traced
API-08 / 09 / 10 · Reads	Services work; CNF document types absent from development
API-11 · Create STO PO	Blocked by configuration — see note
API-12 · Update quantity	Quantity updatable; unit is not

On API-11: we ran the purchase-order create in simulation mode with your document type **ZP06**. SAP replied *“Document type ZP06 not allowed with document category F”*. That is a configuration statement about the development system, not a defect in the interface.

6 Our recommendation

6.1 The question

For the span **shipment creation** → **freight estimate** → **cost document and calculation** → **cost release**, ending immediately before goods issue: does CNF call SAP **once**, or **three times**?

6.2 Why one

1 · Custom development is required either way

Four of six steps have no callable interface. Three endpoints means three custom wrappers, not fewer. The only question is how many doors we cut, not whether we cut any.

2 · SAP has already merged two of the steps

Creating and persisting the cost document is a single SAP operation. Splitting it would invent a boundary that does not exist. This also matches what was agreed on 17 August — costing creation and release merged into one call.

3 · The state lives in SAP, so the caller cannot own it

Only SAP can answer “how far did this get”. Three calls means the orchestrator holding a state machine for data it cannot read. One call that reads those documents and continues from where it finds itself is **safe to retry by design** — the most valuable property this interface can have.

4 · The 10,496

Splitting the steps is survivable — SAP's own implementation survives it at 0.18%. But nothing about three endpoints makes that number smaller, and two things make it larger: a network between the steps, and an orchestrator that cannot see the cost tables.

6.3 What the API returns

```
{ "shipment":      "1100605471",
  "costDocument":  "1100608871",
  "costType":      "Z001",
  "calculated":    true,
  "accountAssigned": true,
  "settled":       true,
  "freightPO":     "6000105831",
  "serviceEntry":  "1003382363",
  "stage":         "READY_FOR_PGI",
  "messages":      [ SAP's own messages ] }
```

Stage-level status, not a simple success or failure. Every step that completed is named with the document it produced. A failure returns the step it stopped at, plus SAP's own message — which is what the functional-versus-technical error split agreed on 17 August requires in order to route errors correctly.

6.4 Three conditions we state alongside it

1. **It is not atomic, and cannot be.** Once the shipment is committed it is committed; recovery is reversal, not rollback. This is equally true of one API or three — but it should be on the record now rather than discovered later. The word “rollback” used in the 17 August readout should be withdrawn.
2. **It is resumable, not fire-and-forget.** Calling it again reads the existing documents, works out what already exists, and continues. This is what prevents duplicate shipments.
3. **The SAP side does not own process state.** Process identity, stage history and replay protection belong to the orchestrator. SAP holds no cross-step process state and cannot undo a committed goods issue however the code is written.

7 What remains open, and what we need

7.1 Open technical questions

Question	Why it matters
Does shipment creation automatically trigger cost document creation?	If yes, the interface we expose gets smaller
Does the freight estimate genuinely create nothing?	The estimate API depends on it being side-effect free
How is cost type determined?	It is mandatory and currently unspecified

7.2 Why we cannot yet demonstrate this in development

The development system has the **configuration** for this process but almost none of the **data**.

Missing	Consequence
No vendor records at all	No transporter can be entered — this alone blocks every step from shipment onward
No material valuation	No goods movement can be posted
No stock, no batches	Nothing to pick or issue
No sales orders or deliveries	No starting point for the chain

Present and usable: shipment types, transportation planning points, routes, cost document types and standard purchase order types. Your CNF-specific document types `ZP06`, `ZNL` and `ZST0` are **not present in the development system**.

A decision we need from you

	Option A — generic	Option B — CNF parity
Uses	Standard SAP document types	Your <code>ZP06</code> / <code>ZNL</code> / <code>ZST0</code> brought into development
Proves	Mechanics, payloads, commit behaviour, retry	Actual CNF behaviour
Does not prove	Anything about your specific document types	—
Speed	Fast, once master data exists	Needs a configuration transport

We recommend A now, B before sign-off. Option A unblocks the technical proof immediately. Option B is required before anyone signs a contract that names your document types. We will never present Option A results as proof of CNF parity.

7.3 What we are asking for

One disposable test chain in the development system: a valued material with stock, a batch-managed variant, a customer and an order, one delivery not yet goods-issued, **at least one transporter**, a route, and freight rate configuration — plus confirmation of which shipment type, planning point and cost document type to use.

And one process answer that no technical inspection can produce: which transaction or job your team actually runs to create and settle shipment costs.

A field-level list of the fifteen items has been prepared and can be handed to the functional and Basis teams directly.

8 Summary

Finding	Consequence
Four of six pre-PGI steps have no callable SAP interface	Custom development is mandatory either way
SAP merges cost creation and persistence internally	Splitting them invents a boundary that does not exist
Chain state lives only in SAP documents	An external orchestrator cannot reliably own it
10,496 documents stranded at 0.18%, inside SAP, with no network	Splitting across a network will not improve on that
Settlement does not technically block goods issue	The sequencing rule is yours to enforce, not SAP's
Cost type is mandatory and varies	A missing field in the current specification
The pattern already runs here in custom code	Feasibility proven; the shape is known

RECOMMENDATION

One resumable pre-PGI API, returning stage-level status and document numbers.

Not atomic — we will say so plainly. Not a thin passthrough — the orchestration logic is real and belongs to the orchestrator. But **one call**, because SAP gives us no honest place to cut it into three.

Prepared from live system evidence in DS4/200 and QS4/700. Supporting detail, document traces, captured interfaces and full status counts are available on request.